Universidad de Valparaíso
CHILE

Prueba de Sistemas

TicketU · Lo que se implementó

Guía del sistema bajo prueba · pruebasdesistemauv.com/ticketu · 9 microservicios + equipo de plataforma

Docente: Natalia Zepeda

Universidad de Valparaíso CHILE

Agenda

Del sistema implementado a sus casos de prueba

01

Diseño vs. implementado

Qué cambió y qué se mantiene

02

El sitio y su arquitectura

Accesos, gateway y broker

03

Cada microservicio

Funciones, API y pantallas

04

Eventos y flujo

Gratuito vs. pagado

05

Plataforma y roles

Componentes comunes y permisos

06

Datos de prueba

Usuarios, eventos, tarjetas, códigos

07

Reglas de negocio

El resultado esperado de sus casos

08

Cómo probar

Apoyos, reutilización y normas

Todo lo que verán aquí ya está funcionando en pruebasdesistemauv.com/ticketu.

Del diseño al sistema implementado

La arquitectura se mantiene: cambia cómo se despliega en el hosting del curso

Lo diseñado

- Un contenedor Docker por microservicio
- Despliegue on-premise en la sala de servidores
- MongoDB: una base de datos por servicio
- Broker de eventos externo (RabbitMQ)
- Contrato OpenAPI publicado por cada servicio

Lo implementado

- Una aplicación Node en el hosting (cPanel) que levanta el gateway y los 9 servicios
- Publicada en pruebasdesistemauv.com/ticketu
- Cada servicio con sus propios datos (carpeta data/<servicio>/)
- Broker de eventos dentro de la aplicación: los servicios siguen sin llamarse entre sí
- Contratos OpenAPI navegables en /ticketu/api/docs/

Lo importante para las pruebas: la lógica, los contratos y la comunicación por eventos son los del diseño. Lo que cambia es la infraestructura.

El sitio implementado

Un solo ambiente de pruebas compartido por los 9 equipos

TicketU:
vende y reserva entradas para eventos de la universidad. Los eventos gratuitos y pagados comparten el mismo flujo de emisión de entradas con código QR, tal como se definió en el proyecto.

Accesos:

- **Sitio:** pruebasdesistemauv.com/ticketu
- **Contratos de API:** /ticketu/api/docs/
- **Estado de los servicios:** /ticketu/api/health
- **APIs:** /ticketu/api/<servicio>/…
- **Pantallas:** rutas con # (ej. #/checkout/<orden>)

Evento gratuito

Ej.: Charla: IA en la universidad

Reservar → la orden queda confirmada al instante → entradas TKU-… con QR, sin pasar por Pagos

Evento pagado

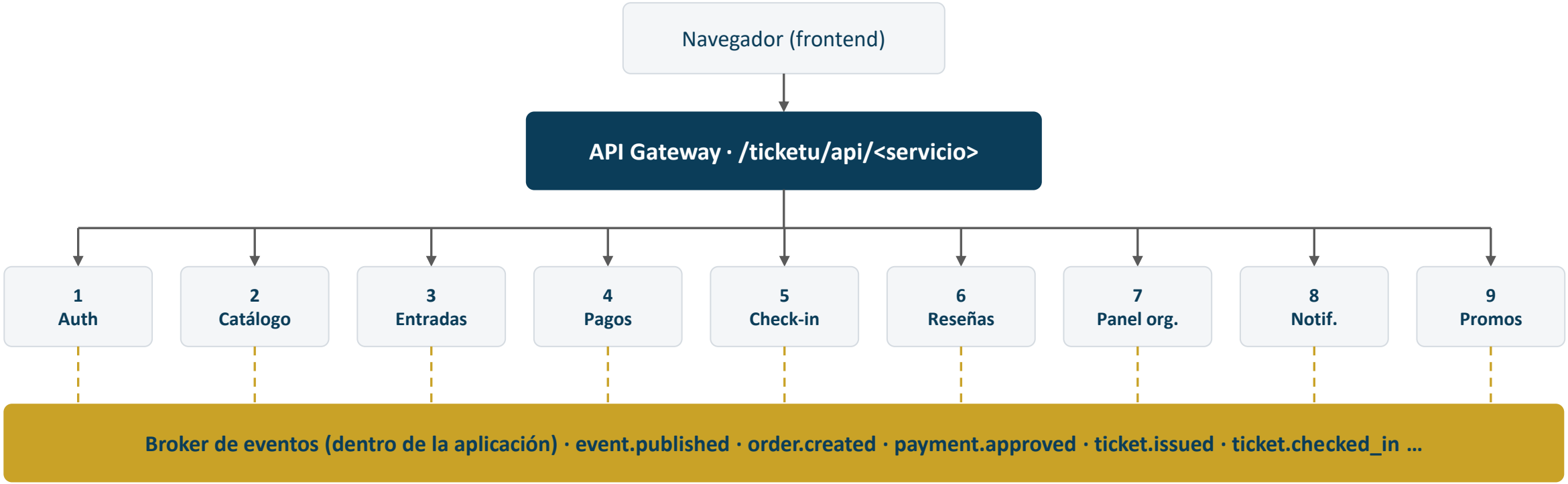
Ej.: Fiesta de generación 2026

Ir a pagar → orden pendiente 15 min → checkout con código de descuento y tarjeta → pago aprobado → entradas con QR

Todos los equipos prueban sobre la misma instancia: lo que crea o cambia un equipo lo ven los demás.

Arquitectura implementada

API Gateway al frente, un microservicio por equipo y broker de eventos entre ellos



Cada servicio guarda solo sus datos y ningún servicio lee los de otro. Un cambio llega a los demás únicamente como evento: Pagos publica payment.approved y Entradas emite las entradas.

Universidad de Valparaíso CHILE

Las dos piezas que unen el sistema

El API Gateway y el broker de eventos: cómo entran las peticiones y cómo se coordinan los servicios

API Gateway · la única puerta de entrada

- Todas las peticiones pasan por /ticketu/api/<servicio>.
- El gateway lee el nombre en la ruta y la dirige al microservicio correcto (/api/pagos → Pagos).
- Aplica controles comunes a todos: límite de solicitudes por minuto (429) y estado de los servicios en /api/health.
- Publica la documentación de cada servicio en /api/docs.
- Marca cada petición con un x-request-id para seguirla en los registros.

Broker de eventos · la coordinación

- Es el canal por donde los servicios se avisan entre sí: publican y escuchan eventos.
- Los servicios nunca se llaman directamente; solo conocen los eventos (event.published, payment.approved, ticket.issued…).
- Un cambio en un servicio llega a los demás como evento: Pagos aprueba y Entradas emite las entradas.
- Así los equipos quedan desacoplados: cada uno trabaja sin depender del código de los otros.
- Es la base de las pruebas de integración: una acción debe reflejarse en los servicios que reaccionan.

El gateway unifica cómo entran las peticiones; el broker coordina a los 9 servicios sin acoplarlos. Juntos convierten servicios independientes en un solo sistema.

Universidad de Valparaíso CHILE

Qué implementa cada microservicio

Backend y fragmento de frontend de cada equipo, tal como quedaron en el sistema

Equipo 1

Auth

Registro, login con token (8 h), bloqueo tras 5 intentos, creación de staff por el admin

Equipo 2

Catálogo de eventos

Eventos publicados y futuros; filtros por texto, categoría, gratis/pagado y fechas; paginación

Equipo 3

Entradas / inventario

Stock, órdenes (máx. 4 por persona), expiración a los 15 min, entradas con QR

Equipo 4

Pagos

Checkout simulado con validación de tarjeta, 3 intentos, reembolso si se cancela el evento

Equipo 5

Check-in

Valida el código en la puerta y lo marca como usado; resumen de ingreso por evento

Equipo 6

Reseñas

Nota 1–5 y comentario, solo para asistentes de eventos ya terminados

Equipo 7

Panel organizador

Crear, editar, publicar y cancelar eventos; dashboard de ventas y asistencia

Equipo 8

Notificaciones

Avisos de reserva, entrada, pago, reembolso y cancelación; recordatorio 24 h antes

Equipo 9

Promociones

Códigos de porcentaje o monto fijo con vigencia, evento y límite de usos

Universidad de Valparaíso CHILE

Dónde probar cada servicio

Ruta de la API (bajo /ticketu/api) y pantallas del frontend de cada equipo

Equipo	Servicio	API	Pantallas (frontend)
1	**Auth**	/api/auth	#/login · #/register · #/admin/users
2	**Catálogo de eventos**	/api/catalog	#/ (inicio con filtros) · #/event/<id>
3	**Entradas / inventario**	/api/tickets	Selector en #/event/<id> · #/my-tickets · #/ticket/<id> · #/orders
4	**Pagos**	/api/payments	#/checkout/<orden>
5	**Check-in**	/api/checkin	#/checkin (staff u organizador)
6	**Reseñas**	/api/reviews	Reseñas en #/event/<id> · #/reviews
7	**Panel organizador**	/api/organizer	#/organizer · #/organizer/new · #/organizer/event/<id>
8	**Notificaciones**	/api/notifications	#/notifications (contador en el menú)
9	**Promociones**	/api/promotions	Código en #/checkout/<orden> · #/promotions

Las pantallas se abren con la ruta que va después del # en la URL del sitio.

Eventos implementados entre servicios

Quién publica cada evento del broker y qué servicios reaccionan

Evento	Lo publica	Reaccionan	Efecto esperado
event.published / updated	Panel organizador	Catálogo, Entradas, Reseñas	Aparece en el catálogo y se habilita la venta
event.cancelled	Panel organizador	Catálogo, Entradas, Pagos, Reseñas, Notif.	Entradas canceladas, pagos reembolsados, aviso
order.created	Entradas (solo pagados)	Pagos, Promociones, Notif.	Se puede pagar y aplicar un código
order.expired	Entradas	Pagos, Promociones, Notif.	Stock liberado; ya no se puede pagar
promotion.applied	Promociones	Pagos	El checkout muestra el nuevo total
payment.approved / rejected	Pagos	Entradas, Panel org., Notif.	Se emiten las entradas o se libera el stock
ticket.issued / cancelled	Entradas	Check-in, Panel org., Notif.	La entrada se puede validar en la puerta
ticket.checked_in	Check-in	Entradas, Reseñas, Panel org.	Entrada usada; habilita la reseña

Base para las pruebas de integración: una acción en un servicio debe reflejarse en los servicios que reaccionan a su evento.

El bus de eventos y el diseño de sus pruebas

Qué cambia respecto a un broker como RabbitMQ: actualicen sus casos de integración

Sí deben probar (efectos de los eventos)

- Verifiquen el efecto del evento consultando el estado por la API (ej.: tras pagar, la orden queda confirmada y se emiten las entradas).
- Esperen o sondeen hasta que el efecto ocurra: la entrega es casi inmediata, pero es asíncrona.
- Cada acción debe producir el resultado esperado en todos los servicios que reaccionan (tabla anterior).
- Flujos completos: cancelar un evento → entradas canceladas, pago reembolsado y aviso al usuario.

No aplica aquí (sí con RabbitMQ)

- Persistencia: los eventos no se guardan; no diseñen pruebas de recuperación tras reiniciar el sistema.
- Reintentos y duplicados: un evento no se reencola ni se entrega dos veces; no prueben reentregas.
- Colas por saturación: no hay encolamiento ni contrapresión si un servicio se satura.
- Inspección del bus: no hay consola de colas; verifiquen siempre por los efectos, no por el broker.

Diseñen la integración sobre los efectos observables (estado final vía API). Lo propio de RabbitMQ —persistencia, reintentos y colas— no aplica en esta versión: no incluyan esos casos.

Dónde queda la información (persistencia)

El sistema no usa MongoDB: cada servicio guarda en archivos. Cómo comprobar que los datos quedan almacenados

Cómo se almacena

- No usa un motor de base de datos tradicional: cada microservicio guarda sus datos en archivos dentro del servidor (data/<servicio>/).
- Cumple el rol que tendría la base de datos del diseño (MongoDB): un almacén propio por servicio.
- La escritura es inmediata y segura: los datos sobreviven a un reinicio de la aplicación.
- Ningún servicio lee el almacén de otro: cada uno guarda lo suyo.
- Los archivos están en el servidor, fuera del alcance del navegador.

Cómo verificar que queda guardado

- Escribir y leer de vuelta: crear un dato (usuario, orden, reseña) y consultarlo con un GET o en la pantalla.
- Entre sesiones: cerrar sesión y volver a entrar, o desde otro navegador, con la misma cuenta: el dato sigue.
- En el tiempo: un dato creado antes sigue disponible después.
- Tras reinicio: reiniciar la app y comprobar que los datos continúan (lo controla el docente).

La información del negocio persiste. Verifíquenla por los efectos —lo que se escribe se recupera después—, no inspeccionando el almacenamiento.

Flujo implementado: evento gratuito vs. pagado

Entradas y Pagos se coordinan por eventos, con los tiempos y límites reales del sistema

1

Seleccionar evento

Máximo 4 entradas por persona y evento.

2

¿Gratuito o pagado?

Gratuito: reserva confirmada al instante, sin pasar por Pagos (Equipo 3). Pagado: orden pendiente por 15 min.

3

Checkout

Solo pagados (Equipos 4 y 9): código de descuento y tarjeta, hasta 3 intentos. Publica payment.approved.

4

Emitir entradas + QR

Equipo 3 emite las entradas y publica ticket.issued.

5

Validación en la puerta

Equipo 5, desde 6 h antes hasta 12 h después del inicio. Luego el Equipo 6 habilita la reseña.

Gratuito: confirmado al instante. Pagado: 15 minutos para pagar y hasta 3 intentos rechazados.

Equipo plataforma: lo implementado

Responsabilidades transversales que ya están funcionando en el sistema

01

API Gateway

Única entrada /ticketu/api/<servicio>. Límite de solicitudes por minuto (responde 429) y /api/health

02

Broker de eventos

Dentro de la aplicación. Entrega cada evento a los servicios suscritos; nunca hay llamadas directas

03

CI y calidad

Repositorio con 365 pruebas unitarias (Jest) y análisis de cobertura en SonarCloud

04

Frontend shell

Menú según el rol y navegación por rutas #; cada equipo monta su fragmento

05

Design system

Componentes comunes y atributos data-testid estables para automatizar pruebas de interfaz

06

Observabilidad

Logs por servicio y un x-request-id en cada respuesta para rastrear una petición

Las pruebas no deberían depender de detalles internos de estos componentes, sino de sus comportamientos visibles.

Roles implementados

El menú y los permisos cambian según el rol con que se ingresa

1

Estudiante

Ve eventos, reserva o compra, paga, ve sus entradas con QR, recibe avisos y reseña

2

Organizador

Crea, publica y cancela sus eventos, ve su dashboard, crea códigos y hace check-in de sus eventos

3

Staff

Valida entradas en la puerta y ve el resumen de ingreso de cada evento

4

Admin

Puede todo lo anterior y crea usuarios con cualquier rol (por ejemplo, staff)

Regla de cuentas:
Estudiante y organizador se registran solos en #/register. Staff y admin solo los crea un administrador. Sin sesión → 401; rol sin permiso → 403.

Datos de prueba: usuarios y eventos

Precargados en el sistema · contraseña de los usuarios: Ticketu2026

Rol	Usuario
Staff	staff@ticketu.cl
Organizador	cealumnos@ticketu.cl
Organizador	deportes@ticketu.cl
Estudiante	ana@ticketu.cl
Estudiante	beto@ticketu.cl
Estudiante	carla@ticketu.cl

Evento precargado	Úsenlo para probar
Charla: IA en la universidad (gratis)	Reserva sin pago
Fiesta de generación 2026 ($8.000)	Compra, códigos y pago
Final campeonato de futsal ($3.000)	Solo 5 entradas: stock agotado
Taller express (gratis, terminado)	Reseñas: beto@ticketu.cl asistió
Seminario de tesis (gratis, iniciado)	Evento iniciado: ya no se vende
Bienvenida mechones (borrador)	No aparece en el catálogo

Para probar el check-in:
creen un evento propio que comience dentro de las próximas horas; el ingreso se habilita 6 h antes del inicio.

Datos de prueba: tarjetas y códigos

El pago es simulado: no se cobra dinero real

Tipo	Valor	Resultado esperado
Tarjeta	**4242 4242 4242 4242**	Pago aprobado (también cualquier número válido por Luhn)
Tarjeta	**4000 0000 0000 0002**	Rechazo 402: fondos insuficientes
Tarjeta	**4000 0000 0000 9995**	Rechazo 402: tarjeta bloqueada
Tarjeta	**4000 0000 0000 0119**	Rechazo 402: error del procesador
Código	**BIENVENIDA**	20 % en eventos pagados del Centro de Alumnos
Código	**FIJO2000**	$2.000 de descuento, solo en la Fiesta de generación
Código	**UNICO**	50 %, un solo uso en total
Código	**GRATIS100**	100 %: el checkout se completa sin tarjeta
Código	**FUTURO / VENCIDO**	Rechazo 422: aún no vigente / expirado

Tarjeta válida: número que pase Luhn, titular de 3 o más letras, mes 1–12, año futuro y CVV de 3 dígitos (4 en Amex).

Reglas de negocio implementadas (1/2)

Implementadas en el sistema: úsenlas como resultado esperado de sus casos

Equipo 1

Auth

- Contraseña ≥ 8, con letra y número
- Correo repetido → 409
- Registro libre: estudiante u organizador
- 5 fallos → bloqueo 15 min (403)

Equipo 2

Catálogo

- Solo eventos publicados y futuros
- Filtros: texto, categoría, tipo y fechas
- page ≥ 1 · pageSize 1 a 100
- Filtro inválido → 400

Equipo 3

Entradas

- 1 a 4 por orden y máx. 4 por persona y evento
- Sin stock, iniciado o cancelado → 409
- Pagado: pendiente 15 min, luego expira
- Solo se cancela una orden pendiente

Equipo 4

Pagos

- Tarjeta: Luhn, vigencia, CVV, titular
- Rechazo → 402 con intentos restantes
- 3er rechazo → orden rechazada
- Pagar dos veces o vencida → 409

Equipo 5

Check-in

- Válida → OK · organizador ajeno → 403
- Usada 409 · no existe 404 · cancelada 410
- Otro evento o fuera de horario → 422
- Ventana: 6 h antes a 12 h después

Equipo 6

Reseñas

- Solo asistentes con check-in
- Solo eventos terminados y no cancelados
- Nota entera 1–5 · comentario ≤ 500
- Una por persona y evento → 409

Estas reglas son la referencia para particiones de equivalencia y valores límite.

Reglas de negocio implementadas (2/2)

Implementadas en el sistema: úsenlas como resultado esperado de sus casos

Equipo 7

Panel organizador

- Título 3–120 · capacidad 1–20.000
- Gratis precio 0 · pagado $500 a $1.000.000
- Inicio futuro; término después del inicio
- Con ventas no cambia precio ni tipo

Equipo 8

Notificaciones

- Aviso al reservar, emitir, pagar o rechazar
- Aviso de reembolso y de evento cancelado
- Recordatorio único 24 h antes
- Contador de no leídas en el menú

Equipo 9

Promociones

- Código 4–20 caracteres (A-Z, 0-9, guion)
- Porcentaje 1–100 o monto fijo
- Vigencia, evento y límite de usos
- Un código por orden y un uso por persona

Transversal

Tiempos

- Pago: 15 min desde la orden
- Bloqueo de login: 15 min
- Sesión: 8 h
- Recordatorio: 24 h antes del evento

Transversal

Límites

- 4 entradas por persona y evento
- 3 intentos de pago rechazados
- 5 intentos de login fallidos
- Comentario de reseña: 500 caracteres

Transversal

Respuestas HTTP

- 400 datos · 401 sin sesión · 403 permiso
- 402 pago rechazado · 404 no existe
- 409 conflicto · 410 cancelada
- 422 regla no cumplida · 429 límite

Universidad de Valparaíso CHILE

Cómo probar el sistema

Apoyos que trae TicketU para pruebas de API, de interfaz y de integración

1

Swagger

/ticketu/api/docs/ · contrato de cada servicio; botón Authorize con el token del login

2

data-testid

Selectores estables: login-email, order-submit, promo-code, pay-submit, checkin-result

3

Estado

/ticketu/api/health · verifiquen que los 9 servicios respondan "ok" antes de probar

4

x-request-id

Encabezado de cada respuesta; inclúyanlo al reportar un defecto

Regla para reportar un defecto:
Incluyan: pasos, datos usados, resultado esperado según la regla implementada, resultado obtenido (código HTTP y mensaje), captura y x-request-id, severidad y prioridad en cada defecto.

¿Sirven los casos de prueba que ya hicieron?

Sí: se reutilizan, ajustándolos a lo que se implementó

Cómo adaptarlos

- Ubiquen el endpoint en Swagger o la pantalla y sus data-testid
- Cambien los datos por los usuarios, tarjetas y códigos de prueba, o creen los suyos
- Preparen las precondiciones por el flujo normal (ej.: Pagos necesita una orden de Entradas)
- Comparen el resultado esperado con las reglas implementadas (diapositivas 17 y 18)

¿Ajuste del caso o defecto?

Ajuste del caso
El caso esperaba "máximo 5 entradas" y la regla implementada es 4: se corrige el resultado esperado del caso.

Defecto del sistema
La regla implementada es 4 y el sistema permite comprar 5: se reporta con la evidencia de la diapositiva anterior.

Una diferencia entre lo que el equipo supuso y la regla implementada es un ajuste del caso, no un defecto.

Normas del ambiente compartido

Un solo TicketU para los 9 equipos, en un hosting compartido con otros sistemas

Datos propios	Creen sus usuarios y eventos; no modifiquen ni cancelen los de otros equipos.
Pruebas de carga	Solo en fecha y hora coordinadas con el profesor.
Cuentas	No compartan contraseñas fuera del curso ni intenten usar la cuenta admin.
Estado	Antes de reportar una caída, revisen /ticketu/api/health.
Reinicio de datos	Lo hace el profesor; avisen si el ambiente queda inconsistente.

Autenticación de la API (para Postman)

TicketU no usa API key: se autentica con un token JWT que se obtiene al iniciar sesión

Flujo del token

- Pedir el token: POST /api/auth/login con el correo y la contraseña; la respuesta trae el campo "token".
- Usarlo: en cada petición, cabecera Authorization: Bearer <token> (en Postman: pestaña Authorization → tipo Bearer Token).
- El token dura 8 horas; luego hay que volver a iniciar sesión.
- Tip: guarden el token en una variable de la colección con un test script en el login y reutilícenlo.

Qué requiere token

- Públicos (sin token): ver catálogo y detalle de evento, disponibilidad, leer reseñas, registrarse, iniciar sesión, /api/health y /api/docs.
- Protegidos (con token): comprar, pagar, check-in, reseñar, panel organizador, promociones, notificaciones y usuarios.
- Sin token en un endpoint protegido → 401.
- Con un rol sin permiso para la acción → 403.

No hay API key: el token JWT se obtiene con login y se envía como Bearer. Sin token en un endpoint protegido, la respuesta es 401.

Próxima entrega

Sobre TicketU, con estas herramientas:

1. **Pruebas de integración (API y contratos)**
   → Postman
2. **Pruebas de sistema (funcionales, manuales)**
   → registro de ejecución en TestLink
3. **Métricas**
   → de la ejecución de pruebas y de los defectos
4. **Reporte de defectos**
   → Mantis

Importante: si sus casos de prueba quedaron desactualizados por los cambios en la implementación, deben ajustarse a lo desarrollado.

TicketU — pruebasdesistemauv.com/ticketu